

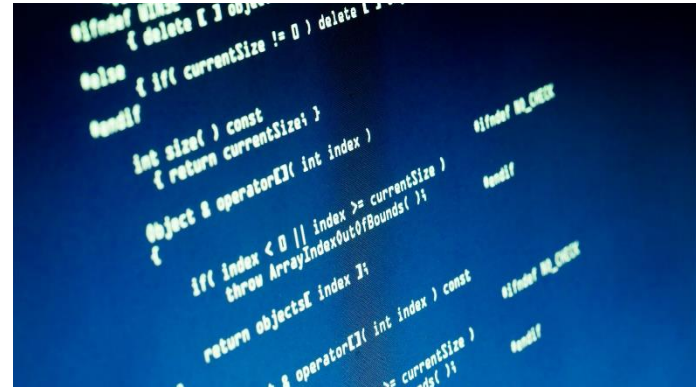
Tehnici de programare - TP



Cursul 3 – Structuri de date abstracte: Stiva și Coada

Ș.I. dr. ing. Cătălin Iapă

catalin.iapa@cs.upt.ro



Structuri de date abstracte

Stiva

Coada

Stiva și Coada în parcurgerea
Grafurilor

Datele în programare

Un program gestionează date.

Datele pot fi:

- **date simple**: int, float, char etc.
- **structuri de date simple**: vector, matrice, struct, union etc.
- **structuri de date abstracte**: stiva, coada, dicționarul etc.

Datele în programare

Structurile de date sunt utilizate pentru a organiza și stoca date într-un mod structurat și eficient.

Acestea sunt esențiale în dezvoltarea aplicațiilor și a algoritmilor, deoarece permit programatorilor să organizeze și să gestioneze seturi de date într-un mod logic și coerent.

Datele în programare

O structură de date poate fi definită ca o colecție de date și o serie de operații care pot fi efectuate pe acestea.

Fiecare tip de structură de date este conceput pentru a rezolva probleme specifice în ceea ce privește stocarea, accesarea și manipularea datelor.

Stiva si Coada

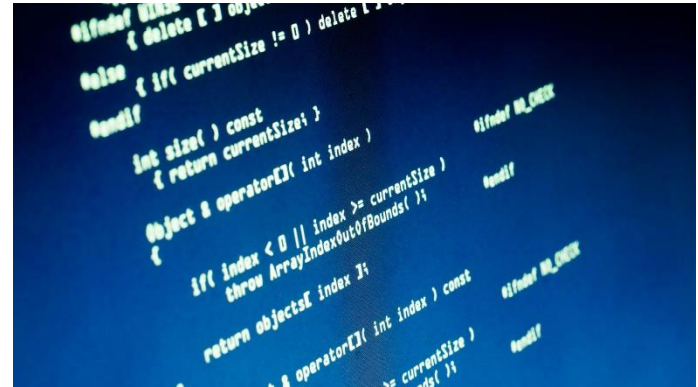
Stiva si coada sunt doua structuri de date importante in programare, utilizate pentru a stoca si organiza date intr-un mod specific.

Stiva este o structura de date LIFO (last in, first out), ceea ce inseamna ca ultimul element adaugat in stiva este primul element extras din stiva.

Coada este o structura de date FIFO (first in, first out), ceea ce inseamna ca primul element adaugat in coada o sa fie primul element extras din coada.

Stiva si Coada





Structuri de date abstracte

Stiva

Coada

Stiva și Coada în parcurgerea
Grafurilor

Stiva

Stiva este
asemanatoare cu o
stiva de farfurii: se
poate adauga o farfurie
noua doar in **varful**
stivei si nu se poate
scoate decat **ultima**
farfurie adaugata.



Exemple de utilizare a Stivei

1. Managementul memoriei în calculatoare:

Sistemele de operare utilizează o stivă pentru a gestiona apelurile de funcții și variabilele locale în cadrul programelor. Fiecare apel de funcție creează un nou cadru de stivă, care este eliminat când funcția se termină.

2. Mecanismul Undo in editoare:

În aplicațiile de editare text sau grafice, operațiile de "undo" (anulare) sunt gestionate de obicei cu o stivă, memorând stările anterioare ale documentului sau imaginii.

Exemple de utilizare a Stivei

3. Navigarea Înapoi în Browser Web:

Când navigați pe internet, fiecare pagină vizitată este pusă pe o stivă. Când apăsați butonul de "înapoi", browserul accesează elementul din vârful stivei.

4. Parcurgeri de Grafuri:

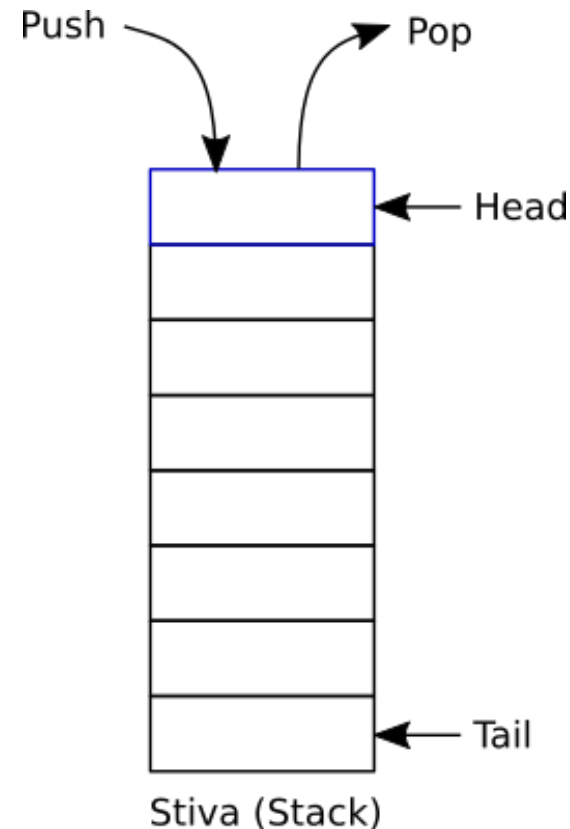
În algoritmi de parcurgere a grafurilor, cum ar fi parcurgerea în adâncime (Depth-First Search), se folosește o stivă pentru a reține nodurile vizitate.

Stiva

Operațiile fundamentale pe o stivă sunt:

- **push** (adaugarea unui element în vârful stivei)
- **pop** (scoaterea unui element din vârful stivei).

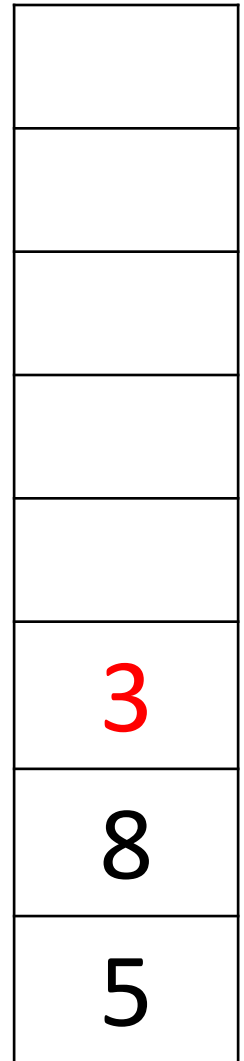
O altă operație comună este **peek**, care permite vizualizarea valorii din vârful stivei fără a o scoate din stivă.



Stiva

De exemplu, să presupunem că avem o stivă goală și **adăugăm (push) elementele 5, 8 și 3** în această ordine.

- **Vârful stivei va fi 3**, deoarece acesta este ultimul element adăugat.
- Dacă scoatem (**pop**) elementul din vârful stivei, vom obține valoarea 3, iar **vârful stivei va fi actualizat la 8**.
- Dacă scoatem (**pop**) încă un element, **vârful stivei va fi actualizat la 5**.



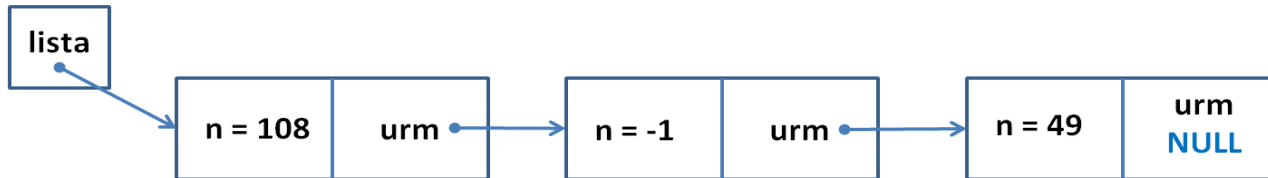
Implementarea unei Stive

O stiva poate fi implementata in C utilizand:

- un vector sau



- o lista simpla inlantuita.



In continuare, vom parcurge cum putem implementa o stiva utilizand un vector.

Implementarea unei Stive

Definim o constanta care sa reprezinte **dimensiunea maxima** a stivei si o variabila care sa reprezinte **varful stivei**. Apoi definim **vectorul care va reprezenta stiva**.

```
#define MAX_SIZE 100  
int top = -1;
```

```
int stack[MAX_SIZE];
```

În acest exemplu am ales să **declaram global** aceste variabile pentru ca funcțiile implementate să aibă o structură cât mai simplă și să fie ușor de urmărit.

Adăugarea unui element în Stivă

Definim o functie care sa **adauge un element** in stiva.

Aceasta functie verifica daca stiva **este deja plina** si afiseaza un mesaj de eroare in caz afirmativ. In caz contrar, **adauga elementul in varful stivei** si incrementeaza varful.

```
void push(int item) {  
    if(top == MAX_SIZE-1) {  
        printf("Stiva este plina\n");  
        return;  
    }  
  
    stack[++top] = item;  
}
```

Ștergerea unui element din Stivă

Definim o functie care sa **extraga** un element din stiva.

Aceasta functie verifica daca **stiva este goala** si afiseaza un mesaj de eroare in caz afirmativ. In caz contrar, **extrage elementul din varful stivei** si **decrementaza varful**.

```
int pop() {  
    if(top == -1) {  
        printf("Stiva este goala\n");  
        return -1;  
    }  
  
    return stack[top--];  
}
```

Afişarea elementelor Stivei

Aceasta functie verifica **daca stiva este goala** si afiseaza un mesaj de eroare in caz afirmativ. In caz contrar, **afiseaza elementele din stiva** de la varful stivei la baza stivei:

```
void display() {  
    if(top == -1) {  
        printf("Stiva este goala\n");  
        return;  
    }  
    printf("Elementele din stiva sunt: ");  
    for(int i=top; i>=0; i--)  
        printf("%d ",stack[i]);  
    printf("\n");  
}
```

Exemplu de utilizare

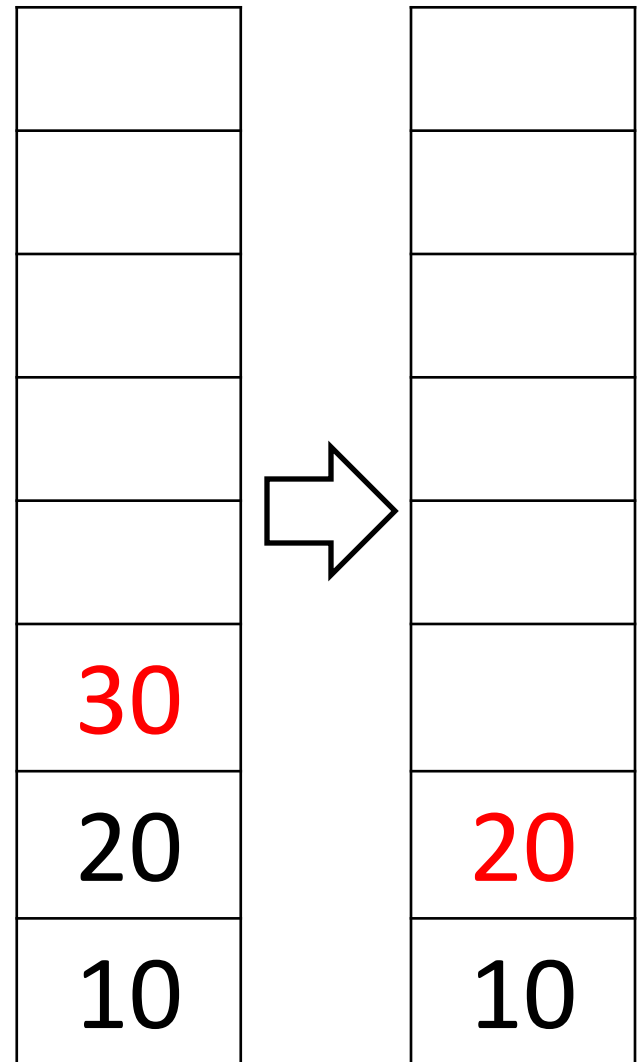
```
int main() {  
    push(10);  
    push(20);  
    push(30);  
    display();  
    printf("Elementul extras din stiva:  
%d\n", pop());  
    display();  
    return 0;  
}
```

OUTPUT:

Elementele din stiva sunt: 30 20 10

Elementul extras din stiva: 30

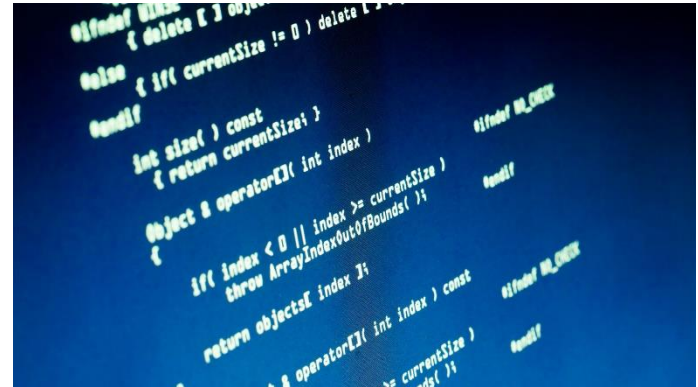
Elementele din stiva sunt: 20 10



Implementarea unei Stive

Aceasta implementare a unei stive in C poate fi imbunatatita prin utilizarea unei **liste simplu inlantuite**, dar ideea de baza ramane aceeaasi.

O sa revenim la **stiva implementata cu ajutorul listelor** cand vom parcurge listele simplu si dublu inlantuite.



Structuri de date abstracte

Stiva

Coada

Stiva și Coada în parcurgerea
Grafurilor

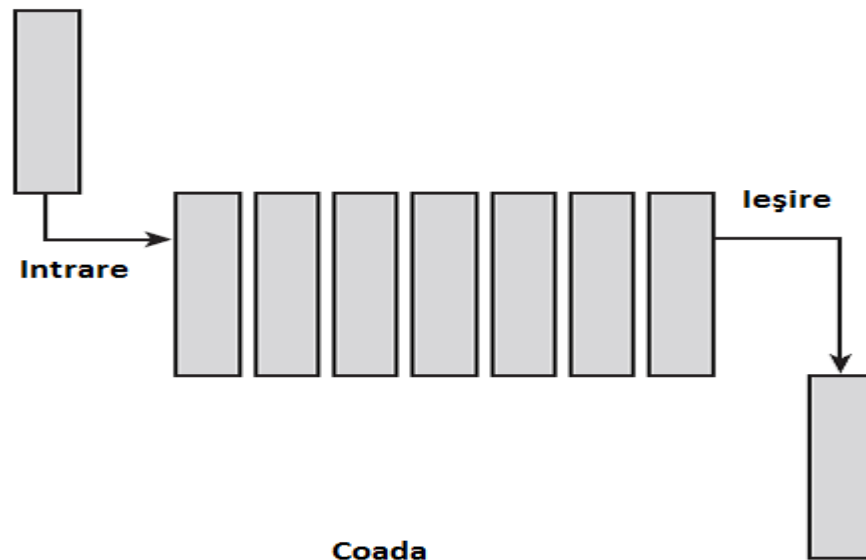
Coada



Coada este
asemanatoare cu o
coada la supermarket:
primul client care
ajunge la coada este
primul care este servit
si paraseste coada.

Coada

In programare, coada este folosita pentru a gestiona sarcinile in **ordinea in care au fost adaugate** sau pentru **a stoca elementele in asteptare** pana cand sunt procesate.



Exemple de utilizare a Cozii

1. Managementul Taskurilor în Sistemele de Operare:

Cozile sunt utilizate pentru a gestiona procesele sau threadurile care așteaptă să fie executate de procesor.

2. Buffer pentru Date în Flux:

În streaming-ul de date, cum ar fi streaming-ul video sau audio, cozile sunt utilizate pentru a gestiona datele care vin în flux, asigurând o redare conformă.

Exemple de utilizare a Cozii

3. Imprimarea Documentelor:

Într-o rețea sau la o imprimantă partajată, documentele trimise spre imprimare sunt puse într-o coadă și procesate în ordinea în care au fost primite.

4. Gestionarea Evenimentelor în Interfețe Grafice:

În programarea GUI, evenimentele generate de utilizator (cum ar fi clicuri de mouse sau apăsări de taste) sunt puse într-o coadă pentru a fi procesate în ordinea în care au apărut.

Coada

Operațiile fundamentale pe o coadă sunt:

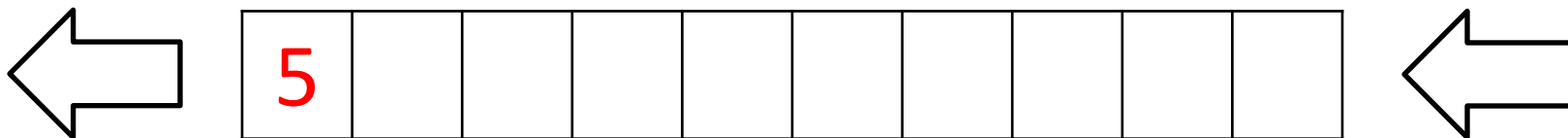
- **enqueue** (adaugarea unui element la capătul cozii) și
- **dequeue** (scoaterea unui element din capul cozii).

O altă operație comună este **peek**, care permite vizualizarea valorii din capul cozii fără a o scoate din coadă.

Coada

De exemplu, să presupunem că avem o coadă goală și adăugăm (**enqueue**) elementele 5, 8 și 3 în această ordine. Capul cozii va fi 5, iar coada cozii va fi 3.

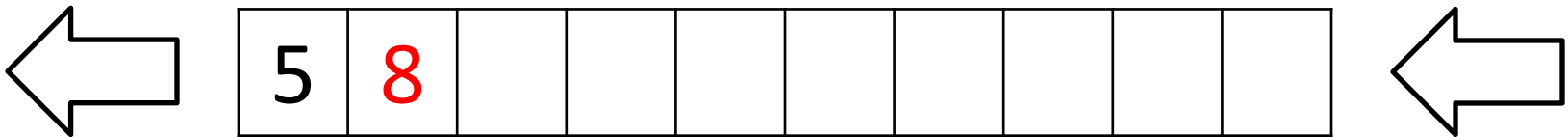
- Dacă scoatem (**dequeue**) elementul din capul cozii, vom obține valoarea 5, iar capul cozii va fi actualizat la 8.
- Dacă scoatem (**dequeue**) încă un element, capul cozii va fi actualizat la 3.



Coada

De exemplu, să presupunem că avem o coadă goală și adăugăm (**enqueue**) elementele 5, 8 și 3 în această ordine. Capul cozii va fi 5, iar coada cozii va fi 3.

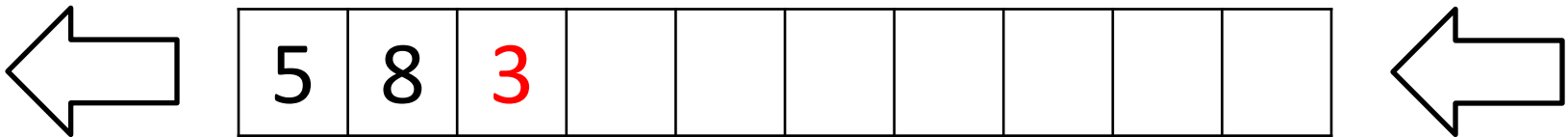
- Dacă scoatem (**dequeue**) elementul din capul cozii, vom obține valoarea 5, iar capul cozii va fi actualizat la 8.
- Dacă scoatem (**dequeue**) încă un element, capul cozii va fi actualizat la 3.



Coada

De exemplu, să presupunem că avem o coadă goală și adăugăm (**enqueue**) elementele 5, 8 și 3 în această ordine. Capul cozii va fi 5, iar coada cozii va fi 3.

- Dacă scoatem (**dequeue**) elementul din capul cozii, vom obține valoarea 5, iar capul cozii va fi actualizat la 8.
- Dacă scoatem (**dequeue**) încă un element, capul cozii va fi actualizat la 3.



Coada

De exemplu, să presupunem că avem o coadă goală și adăugăm (**enqueue**) elementele 5, 8 și 3 în această ordine. Capul cozii va fi 5, iar coada cozii va fi 3.

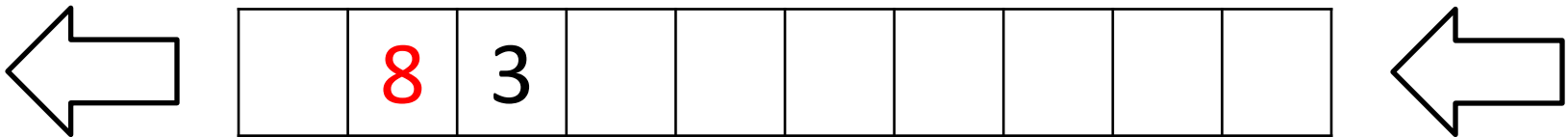
- Dacă scoatem (**dequeue**) elementul din capul cozii, vom obține valoarea 5, iar capul cozii va fi actualizat la 8.
- Dacă scoatem (**dequeue**) încă un element, capul cozii va fi actualizat la 3.



Coada

De exemplu, să presupunem că avem o coadă goală și adăugăm (**enqueue**) elementele 5, 8 și 3 în această ordine. Capul cozii va fi 5, iar coada cozii va fi 3.

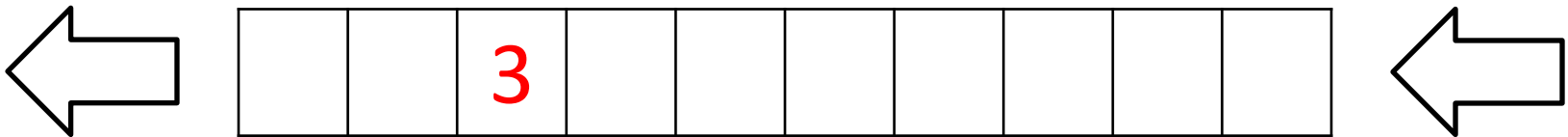
- Dacă scoatem (**dequeue**) elementul din capul cozii, vom obține valoarea 5, iar capul cozii va fi actualizat la 8.
- Dacă scoatem (**dequeue**) încă un element, capul cozii va fi actualizat la 3.



Coada

De exemplu, să presupunem că avem o coadă goală și adăugăm (**enqueue**) elementele 5, 8 și 3 în această ordine. Capul cozii va fi 5, iar coada cozii va fi 3.

- Dacă scoatem (**dequeue**) elementul din capul cozii, vom obține valoarea 5, iar capul cozii va fi actualizat la 8.
- Dacă scoatem (**dequeue**) încă un element, capul cozii va fi actualizat la 3.



Implementarea Cozii în C

La fel ca și la stivă, o coada poate fi implementată în C utilizând

- un vector sau
- o listă simplă înlantuită.

În continuare, vom parcurge cum se poate implementa o coadă utilizând un vector.

Implementarea Cozii în C

Definim o constanta care sa reprezinte **dimensiunea maxima a cozii** si doua variabile care sa reprezinte **inceputul si sfarsitul cozii**. Definim **vectorul** care va reprezenta coada.

```
#define MAX_SIZE 100  
int front = -1, rear = -1;
```

```
int queue[MAX_SIZE];
```

În acest exemplu am ales să **declaram global** aceste variabile pentru ca funcțiile implementate să aibă o structură cât mai simplă și să fie ușor de urmărit.

Adăugarea unui element în Coadă

Definim o functie care sa **adauge** un element in coada. Aceasta functie **verifica daca coada este deja plina** si afiseaza un mesaj de eroare in caz afirmativ. In caz contrar, **adauga elementul in sfarsitul cozii** si actualizeaza variabila "rear". In cazul in care coada este goala, actualizeaza si variabila "front".

```
void enqueue(int item) {  
    if(rear == MAX_SIZE-1) {  
        printf("Coadă este plina\n");  
        return;  
    }  
    if(front == -1) front = 0;  
    queue[++rear] = item;  
}
```

Ștergerea unui element din Coadă

Definim o functie care sa extraga un element din coada. Aceasta functie **verifica daca coada este goala** sau daca variabila "front" depaseste variabila "rear" si afiseaza un mesaj de eroare in caz afirmativ. In caz contrar, **extrage elementul din inceputul cozii** si incrementeaza variabila "front".

```
int dequeue() {  
    if(front == -1 || front > rear) {  
        printf("Coadă este goală\n");  
        return -1;  
    }  
    return queue[front++];  
}
```

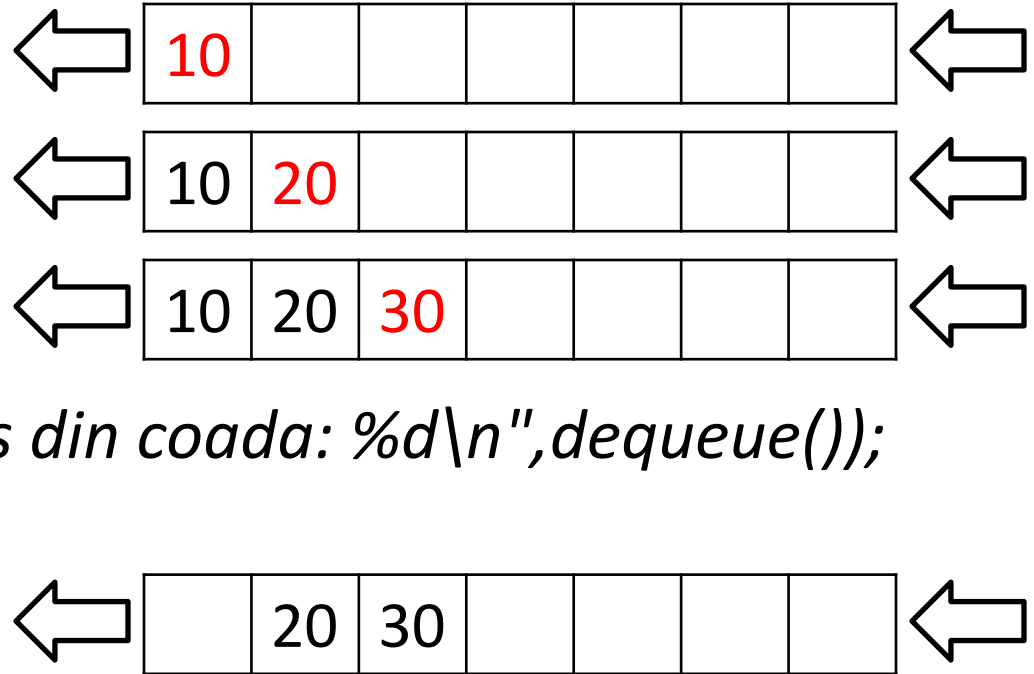
Ștergerea unui element din Coadă

Definim o funcție care sa **afișeze elementele** din coadă, pentru a putea **vizualiza oricând conținutul cozii**.

```
void display() {  
    if(front == -1 || front > rear) {  
        printf("Coadă este goală\n");  
        return;  
    }  
    printf("Elementele din coadă sunt: ");  
    for(int i=front; i<=rear; i++)  
        printf("%d ",queue[i]);  
    printf("\n");  
}
```

Exemplu de utilizare

```
int main() {  
    enqueue(10);  
    enqueue(20);  
    enqueue(30);  
    display();  
    printf("Elementul extras din coada: %d\n", dequeue());  
    display();  
    return 0;  
}
```



OUTPUT:

Elementele din coada sunt: 10 20 30

Elementul extras din coada: 10

Elementele din coada sunt: 20 30

Implementarea unei Cozi

Aceasta implementare a unei cozi in C poate fi imbunatatita prin utilizarea unei **liste simplu sau dublu inlantuite**, dar ideea de baza ramane aceeaasi.

O sa revenim la **coada implementata cu ajutorul listelor** cand vom parcurge listele simplu si dublu inlantuite.

Cozi "hardware"

O coadă poate fi utilizată într-un **buffer hardware** pentru a gestiona datele care sunt transmise între dispozitivele hardware sau software.

Un **buffer** este o zonă de memorie temporară care este utilizată pentru a stoca datele înainte de a fi transmise mai departe la dispozitivul receptor.

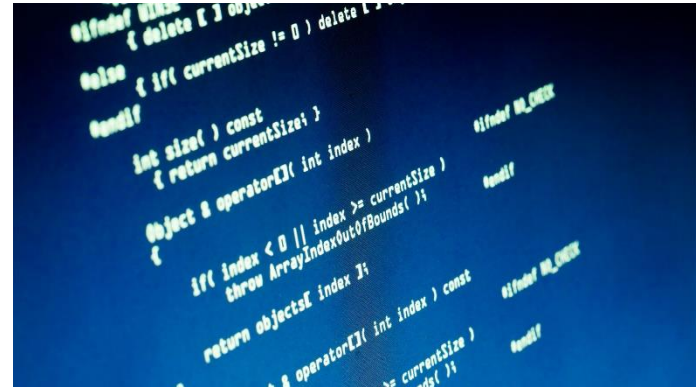
Un buffer poate fi reprezentat de **un registru, o memorie cache sau un alt tip de memorie.**

Cozi "hardware"

În cazul unui buffer hardware, datele sunt **stocate într-o coadă în ordinea în care sunt primite de la sursă**. Apoi, aceste date sunt transferate în buffer și ulterior transmise la dispozitivul receptor.

Utilizarea unei cozi într-un buffer hardware este utilă în situațiile în care **viteza de transmitere a datelor este mai mare decât viteza de procesare a acestora**.

În astfel de cazuri, o coadă poate fi utilizată pentru **a stoca temporar datele și pentru a evita pierderea acestora**.



Structuri de date abstracte

Stiva

Coada

**Stiva și Coada în parcurgerea
Grafurilor**

Stiva și Coada în parcurgerea Grafurilor

Grafurile se pot parcurge

- în adâncime (Depth-First Search - DFS) sau
- prin cuprindere (Breadth-First Search - BFS)

Pentru algoritmul de parcurgere în adâncime se poate utiliza o stivă, iar pentru algoritmul de parcurgere în lățime o coadă.

Stiva în Algoritmi de Parcurgere în Adâncime

Parcurgerea în Adâncime (Depth-First Search - DFS) este un algoritm folosit pentru a parcurge sau căuta elemente într-un graf (sau arbore).

Parcurgerea începe de la un nod și explorează cât mai adânc posibil de-a lungul fiecărei muchii.

Rolul Stivei: Stiva este folosită pentru a ține minte nodurile care trebuie vizitate pe măsură ce algoritmul se aprofundează în structura de date. În implementarea iterativă a DFS, când un nod este vizitat, vecinii săi nevizitați sunt 'pushed' în stivă pentru vizite viitoare.

Coada în Algoritmi de Parcurgere în Lățime (BFS)

Parcurgerea în Lățime – sau parcurgerea prin cuprindere - (Breadth-First Search - BFS) este un alt algoritm pentru parcurgerea sau căutarea elementelor într-un graf (sau arbore).

Începe de la un nod din graf și explorează toți vecinii acestuia, apoi toți vecinii vecinilor și așa mai departe.

Rolul Cozii: Coada este folosită pentru a gestiona ordinea în care nodurile sunt vizitate. Pe măsură ce fiecare nod este procesat, vecinii săi sunt adăugați în coadă, asigurându-se că sunt vizitați în ordinea în care au fost descoperiți.

Vă mulțumesc!